

第12讲

GPT 到底是什么结构？

Encoder、Decoder、Encoder-Decoder 与 Decoder-only

本讲目标

学完后，你应该能区分 Encoder-only、Encoder-Decoder 和 Decoder-only；理解 BERT、T5、GPT 为什么采用不同信息流；解释 Causal Mask、Cross-Attention、自回归生成；并能画出一个 GPT 类语言模型从 Token 到 next-token probability 的完整主干。

Transformer 的三大家族



它们共享 Transformer Block 的基本零件，但信息可见性、训练任务和使用方式不同。

先记一句

Encoder 擅长“把整段看懂”；Decoder 擅长“按顺序往后写”；Encoder-Decoder 则是“先读懂输入，再条件生成输出”。

1. GPT 的名字到底是什么意思？

GPT = Generative Pre-trained Transformer。三个词分别对应：Generative（生成式）、Pre-trained（先在海量文本上预训练）、Transformer（采用 Transformer 架构）。

词	直译	真正含义
Generative	生成式	目标不是只做分类，而是能够连续产生新 token
Pre-trained	预训练	先通过大规模自监督任务学通用语言规律，再适配具体使用方式
Transformer	Transformer	核心计算骨干由 Transformer Block 堆叠构成

常见误区

“GPT” 不是 “聊天软件” 的名字。GPT 是模型路线；ChatGPT 是围绕模型构建的交互式产品，还包含系统提示、工具、记忆、界面等系统层。

2. Transformer 原论文其实不是 GPT 结构

2017 年 Transformer 最初用于机器翻译，经典结构是 Encoder-Decoder：左边 Encoder 读源语言，右边 Decoder 逐步生成目标语言。

- Encoder 内部使用双向 Self-Attention;
- Decoder 内部使用 Causal Self-Attention;
- Decoder 还有 Cross-Attention，用于读取 Encoder 的输出。

为什么这很重要

很多初学者把“Transformer”直接等同于“GPT”。更准确地说：GPT 是 Transformer 家族中的 Decoder-only 分支。

3. 三大家族先整体区分

Transformer 的三大家族

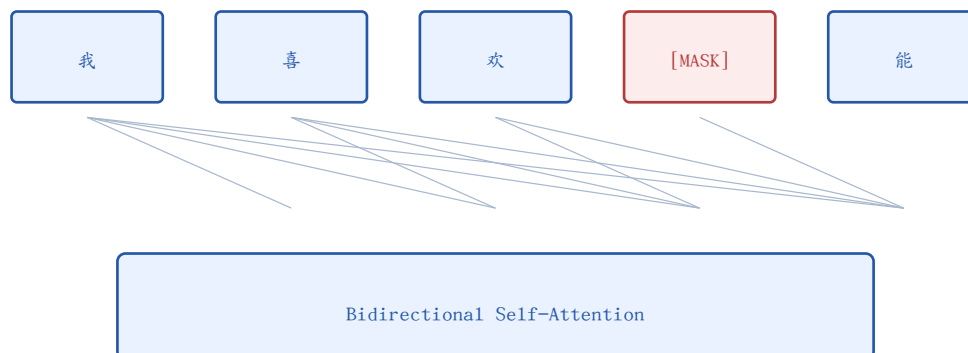


它们共享 Transformer Block 的基本零件, 但信息可见性、训练任务和使用方式不同。

路线	代表	Self-Attention 可见性	典型训练思想	天然强项
Encoder-only	BERT 类	左右文都可见	遮住一些 token, 再恢复	理解 / 表示
Encoder-Decoder	T5 类	Encoder 双向; Decoder 因果	输入→输出条件生成	翻译 / 摘要 / 条件生成
Decoder-only	GPT 类	只能看左侧历史	预测下一个 token	开放式生成 / 对话 / 代码

4. Encoder-only 到底在干什么？

Encoder-only: 整段输入一起“读”，每个位置都能看左右文



典型预训练任务: Masked Language Modeling – 用左右文恢复被遮住的 token。

Encoder 的目标是为每个位置构造一个充分融合左右文的上下文化表示。例如句子中某个词的含义可能依赖前文，也依赖后文，因此允许双向读取很自然。

直觉

Encoder 更像“读完整篇材料后再做理解题”。它并不是天然按 token 一个接一个往后生成文本。

5. BERT 为什么使用 [MASK]?

同一句话，BERT 类与 GPT 类在训练时“看到的世界”不同



不是“谁更高级”，而是预训练目标和信息流不同，因此天然适合的任务形态不同。

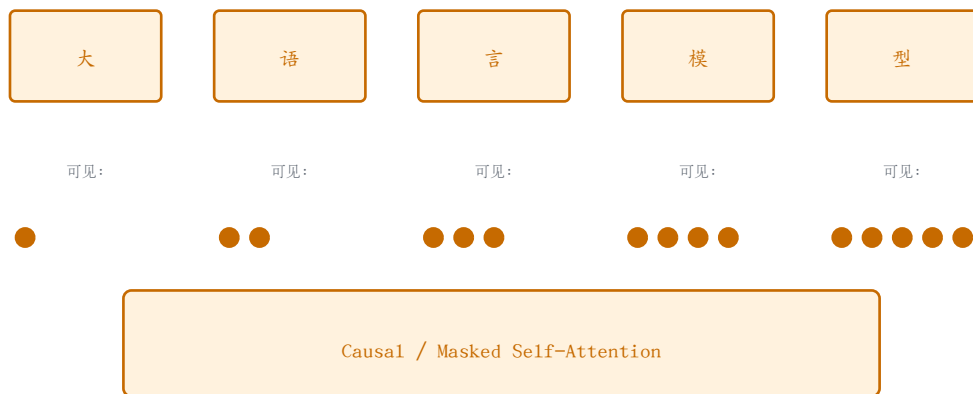
BERT 的经典预训练任务之一是 Masked Language Modeling (MLM)：随机遮住一部分 token，让模型用上下文恢复它。因为答案位置可以看左右文，模型被鼓励学习深度双向语义表示。

注意

真实 BERT 预训练细节还包括采样比例等设计。零基础阶段先抓住“遮住再恢复 + 双向上下文”这条主线。

6. Decoder-only 到底在干什么？

Decoder-only: 每个位置只能看自己和左边，不能偷看未来



典型训练目标: 给定前缀，预测下一个 token；训练时一次算很多位置，生成时逐步追加。

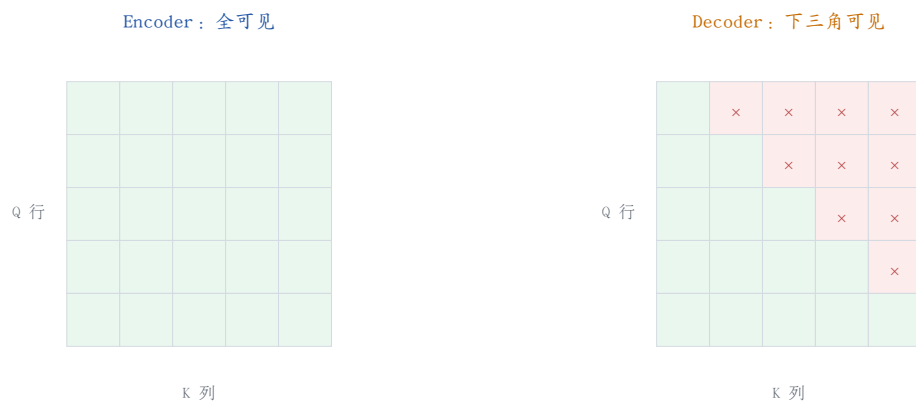
Decoder-only 模型的每个位置都必须遵守“不能看未来”的规则。这样第 t 个位置的表示只依赖 $1 \dots t$ 的 token，才能公平地用来预测第 $t+1$ 个 token。

一句话

GPT 的基本游戏规则：只给过去，不给未来；根据已经看到的前缀预测下一个 token。

7. Causal Mask 为什么是 GPT 的关键？

Encoder 与 Decoder 的关键差别之一：Attention 可见性矩阵



Causal Mask 的本质：未来位置的 score 在 Softmax 前被屏蔽，使生成时不能使用“答案”。

训练时，整段文本可以一次送进 GPU 并行计算，但 Causal Mask 会让每个位置只能访问左侧。于是我们既能并行训练很多位置，又不泄露未来答案。

底层细节

通常不是“把未来 token 从输入里删除”，而是在 Attention score 上把未来位置设成一个极小值（可理解为 $-\infty$ ），经过 Softmax 后权重接近 0。

8. Decoder-only 为什么叫“Decoder”，明明没有 Encoder？

“Decoder”这个名字来自原始 Encoder-Decoder Transformer。GPT 保留了 Decoder 里最关键的 Causal Self-Attention + FFN + Residual + Norm 结构，但去掉了依赖 Encoder 的 Cross-Attention，于是形成 Decoder-only。

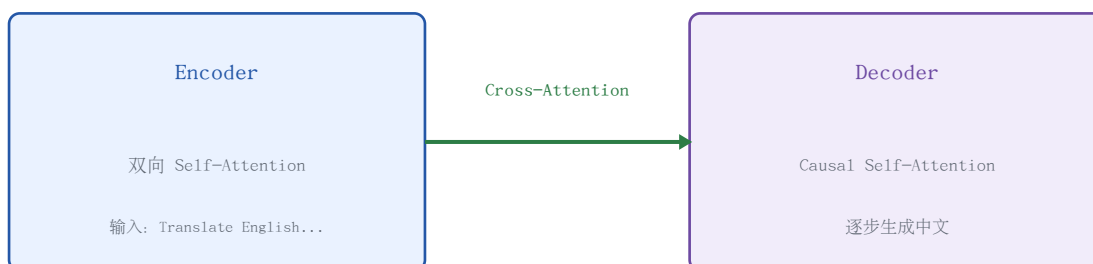
组件	原始 Transformer Decoder	GPT 类 Decoder-only
Causal Self-Attention	有	有
Cross-Attention to Encoder	有	通常没有
FFN / MLP	有	有
Residual / Norm	有	有
是否依赖 Encoder 输出	是	否

误区

Decoder-only 不等于“只有一层 Decoder”。它通常是几十甚至更多个 Decoder-style Transformer Block 堆叠。

9. Encoder-Decoder：为什么要把“读”和“写”分开？

Encoder-Decoder：输入先被编码，Decoder 一边生成一边读取 Encoder 表示



Decoder 的 Query 来自自己；Key/Value 来自 Encoder 输出。

典型用途：翻译、摘要、条件生成。T5 是代表路线之一。

在翻译场景中，源句和目标句是两套序列。Encoder 可以一次性充分理解源句；Decoder 生成每个目标 token 时，再通过 Cross-Attention 动态读取源句表示。

类比

像一个口译员：先听懂整句英语（Encoder），再一句句输出中文（Decoder）；输出过程中仍不断回看自己对原句的理解（Cross-Attention）。

10. Cross-Attention 与 Self-Attention 到底差在哪？

	Query 来自	Key / Value 来自	核心目的
Self-Attention	当前序列	同一个序列	序列内部交流
Cross-Attention	Decoder 当前状态	Encoder 输出	从另一套表示中检索相关信息

因此 Cross-Attention 的关键不在公式完全不同，而在于 Q 与 K/V 的来源不同。这也是第8讲“Q 找 K，权重读 V”的直接延伸。

科研连接

Agent 中“当前任务状态去检索外部记忆”在抽象上也有 Query→Key/Value 的味道，但不能把 Agent 检索简单等同于神经网络 Cross-Attention。

11. GPT 训练：一个句子可以同时提供很多监督信号

假设文本是：我 喜欢 学习 AI。Decoder-only 训练会形成多个“前缀→下一个 token”的训练对：

给模型看的前缀	目标
我	喜欢
我 喜欢	学习
我 喜欢 学习	AI

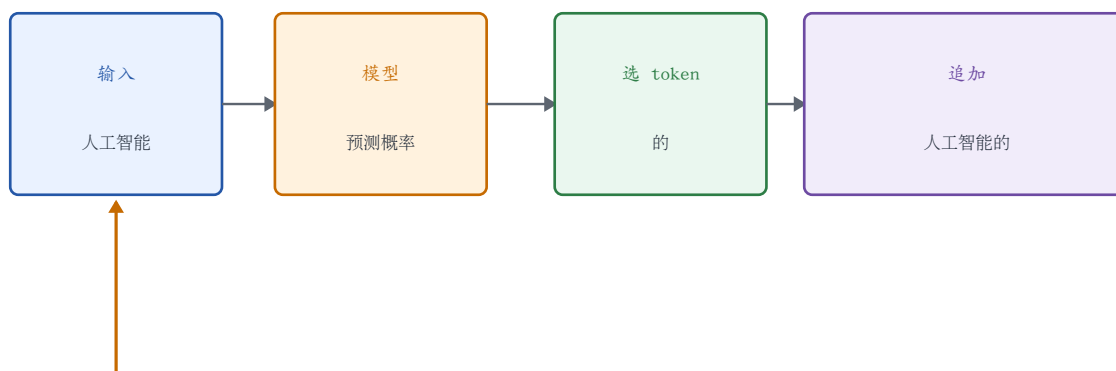
由于 Causal Mask，同一次前向传播可以并行计算多个位置的 next-token loss。这是“自回归任务”与 GPU 并行训练能够同时成立的关键。

训练与生成不同

训练时已知整段正确文本，可以一次并行算很多位置；真正生成时，下一个 token 尚不存在，所以通常必须逐步循环。

12. GPT 推理：为什么一个 token 一个 token 往外出？

GPT 的生成循环：同一个 Decoder-only 模型被反复调用



把新 token 放回上下文，继续预测下一个

这就是 autoregressive（自回归）：前面已经生成的内容，成为后续生成的条件。

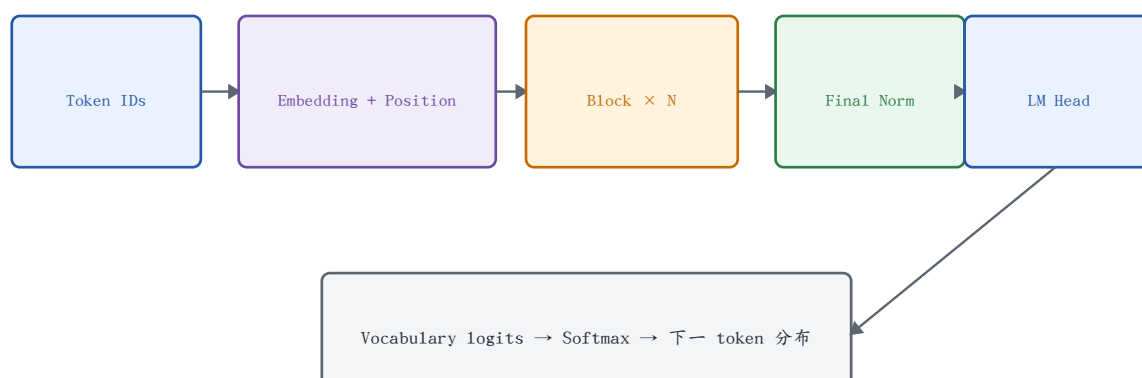
在生成阶段，每次只能先得到下一个 token，追加到上下文后，才能继续得到再下一个。这种“输出又成为下一步输入”的结构叫 Autoregressive（自回归）。

和第1讲接上了

第1讲的“预测一个 token → 接回上下文 → 再预测”并不是产品层的动画效果，而是 Decoder-only 自回归生成机制的直接结果。

13. 一个 GPT 类模型的完整主干

一个 GPT 类模型从 token 到 next-token logits 的完整主干



LM Head 把 d_{model} 维 hidden state 映射到 $|V|$ 个词表分数。

- Tokenizer 把文字变成 Token IDs;
- Embedding 把 ID 变成向量，并注入位置信息;
- N 个 Decoder-style Transformer Blocks 反复改写表示;
- Final Norm 稳定最终表示;
- LM Head 把 hidden state 投影到整个词表，得到 logits;
- Softmax / sampling 决定下一 token。

14. LM Head 是什么？为什么最后突然回到词表？

Transformer Block 内部一直在 d_{model} 维空间里处理向量，但最终任务是“在词表 V 中选哪个 token”。因此需要一个线性层把 d_{model} 维向量投影到 $|V|$ 个分数：

$$\text{logits} = h \cdot W_{\text{vocab}} + b$$

如果词表有 100,000 个 token，那么最后一个位置会得到约 100,000 个 logits。经过 Softmax 后，它们变成一个概率分布。

Weight Tying

不少语言模型会让输入 Embedding 矩阵与输出 LM Head

权重共享或关联，以减少参数并利用输入/输出词表结构。具体模型实现可能不同。

15. 为什么 GPT 类模型特别适合“统一成文本接口”？

因为 Decoder-only 只要求一件事：给我一段前缀，我继续写。于是大量任务都可以重写成文本续写：

任务	可以怎样变成“继续写”
问答	问题：... 回答：
翻译	English: ... Chinese:
代码	需求：... 代码：
分类	文本：... 标签：
工具调用	用户请求：... 工具调用： {...}

这就是 Prompt 的根本便利

Prompt Engineering 的基础，不是模型突然有一个“提示词模块”，而是很多任务都被编码进同一个 token 序列，交给同一个 next-token predictor。

16. 为什么现代通用 LLM 大量走 Decoder-only?

为什么现代通用 LLM 大量采用 Decoder-only?



注意：这不是证明 Encoder 或 Encoder-Decoder “过时”。它们仍在检索、表示学习、翻译等场景中有价值。

Decoder-only 的优势更多是“统一性 + 可扩展性 + 生成天然匹配 + 工程生态成熟”的组合结果，而不是一个简单理论定理。不同任务、不同规模、不同历史阶段仍可能采用其他路线。

不要绝对化

“Decoder-only 成为通用 LLM 主流”不等于“Encoder 已无价值”或“Encoder-Decoder 一定更差”。架构选择始终与任务、数据、算力和系统目标有关。

17. BERT、T5、GPT：用同一张表彻底区分

维度	BERT 类	T5 类	GPT 类
主结构	Encoder-only	Encoder-Decoder	Decoder-only
输入可见性	双向	Encoder 双向; Decoder 因果	因果
主要预训练直觉	恢复被遮文本	文本到文本	预测下一个 token
生成能力	不是天然主接口	天然	天然
是否 Cross-Attention	否	是	通常否
典型使用直觉	读懂 / 表示	有输入条件的生成	开放式续写 / 对话

最小闭环

你不需要背所有模型细节，但必须能从“信息能看哪里、训练目标是什么、输出怎么产生”三个角度判断它属于哪一类。

18. 一个极容易混淆的问题：双向是不是一定比单向“更懂”？

不能这样简单比较。Encoder 的双向可见性使它在构造输入表示时能直接利用左右文；Decoder-only 则受因果约束，但可以通过大量预训练、深层网络和生成过程形成非常强的通用能力。

- “能看未来”是训练信息流设计，不等于模型整体智力评分；
- Decoder-only 的每个 token 表示不能访问未来，但用户提供的完整 Prompt 已经都在“过去上下文”里；
- 任务适配、模型规模、数据质量和训练方式都会影响最终能力。

科研习惯

看到架构差异时，不要直接推导“谁一定更强”；先明确任务、信息约束、训练目标和评测条件。

19. GPT 与 ChatGPT / Agent System 是什么层级关系？

层级	是什么	例子
基础模型	Token→概率分布的神经网络	GPT 类 Decoder-only LLM
对话模型/产品	基础模型 + 指令对齐 + 系统提示 + UI 等	聊天助手
Agent	LLM + 状态 + 工具 + 记忆 + 控制循环	Coding Agent / Research Agent
Multi-Agent System	多个 Agent 的交互组织	协作、辩论、分工系统

这对你的 Agent Systems 学习非常关键：Decoder-only Transformer 是“认知内核”的一种实现，而 Agent System 是更高层的系统结构。

不要层级错位

研究 Agent 的通信、工具、记忆失败时，问题可能来自模型，也可能来自系统编排、上下文构造、环境反馈；不能全部归因于 Transformer 本身。

20. 常见误区集中纠正

误区	正确理解
“Transformer = GPT”	GPT 只是 Transformer 的 Decoder-only 分支。
“Decoder-only 没有 Encoder，所以不能理解输入”	它会把 Prompt 当作历史上下文进行处理，只是不采用独立 Encoder 模块。
“Causal Mask 让模型一次只看一个 token”	训练时仍可并行计算多个位置，只是每个位置不能访问未来。
“BERT 不会生成，所以不智能”	它的架构和训练目标更偏表示理解，不应拿开放式生成当唯一智力指标。
“Cross-Attention 就是 RAG”	二者抽象上都有查询外部信息，但机制层级完全不同。

21. 小实验：亲眼看 Encoder mask 与 GPT causal mask

材料包中的 04_architecture_masks_demo.py 不需要 PyTorch，只用 NumPy。它会构造同一组 Q/K/V，然后分别使用：

- Encoder：全可见 attention；
- Decoder-only：causal mask；
- 打印两个 attention matrix，观察未来位置如何被压成 0；
- 最后演示一个极简“前缀→下一个 token→追加”的自回归循环。

运行命令：

```
python 04_architecture_masks_demo.py
```

实验目的

不是复现 GPT，而是把“架构差异最终落在信息可见性和生成循环上”变成可以观察的数字。

22. 本讲最低闭环：请你能自己画出这三幅图

- Encoder-only: 输入 → 双向 Self-Attention Blocks → 表示 / 任务头;
- Encoder-Decoder: 输入 → Encoder → Cross-Attention → Causal Decoder → 输出;
- Decoder-only: Prompt → Causal Transformer Blocks → LM Head → next token → 追加 → 循环。

如果能做到

你已经跨过“知道 GPT 是 Transformer”这个模糊阶段，进入了“能解释 GPT 为什么这样搭、为什么这样训练、为什么这样生成”的结构理解阶段。

23. 与下一讲衔接：结构已经懂了，模型究竟怎样被“喂大”？

现在我们已经知道一个 GPT 类模型内部怎样流动，但还有一个更现实的问题：这些数十亿、数百亿参数究竟通过什么数据和训练任务学出能力？

- 预训练语料是什么？
- next-token prediction 如何在海量文本上形成训练信号？
- 为什么模型会学到语法、事实、代码和某些推理模式？
- 参数、数据、算力为什么会形成 Scaling Law？

下一讲

第13讲将进入 Pretraining：大语言模型究竟是怎样预训练出来的？